



bkhack!

A computer-science social news website for HCMUT

Le Nguyen Gia Bao ✉ bao.le108101@hcmut.edu.vn

Capstone DATN-328

HCMUT-VNUHCM – 29/12/2025



Content

1. About
2. Motivation
3. Related works
4. Analysis
5. Design
6. Realization
7. Timeline



About

Teachers

- Lecturer 1: Dr. Truong Tuan Anh
- Lecturer 2: ThS. Nguyen Minh Tam
- Defense: ThS. Mai Duc Trung



Members

Group: DAGD1-514

ID	Name	Contribution
2210216	Le Nguyen Gia Bao	100%
2252887	Ho Gia Tuong	100%
2252295	Le Cong Minh Khang	100%

Motivation

- Knowledge fragmentation
- Knowledge degradation



Goals

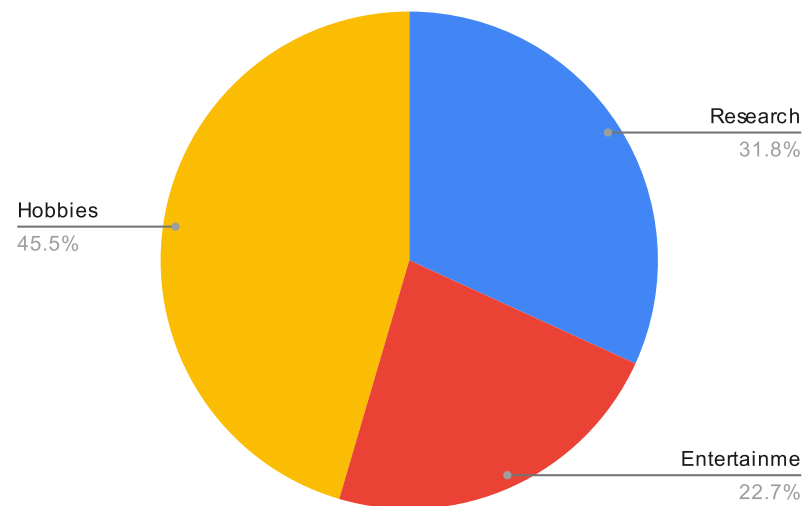
Develop a computer science social news website

- Sharing
- Learn and discover new stuff
- Fact-checked

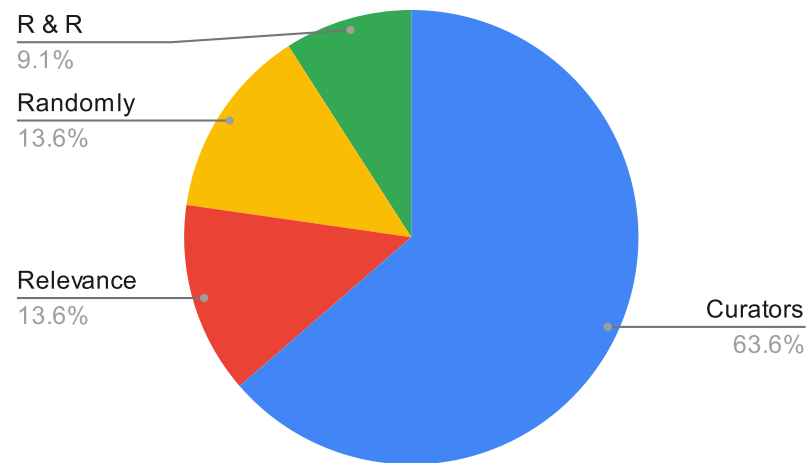
Survey



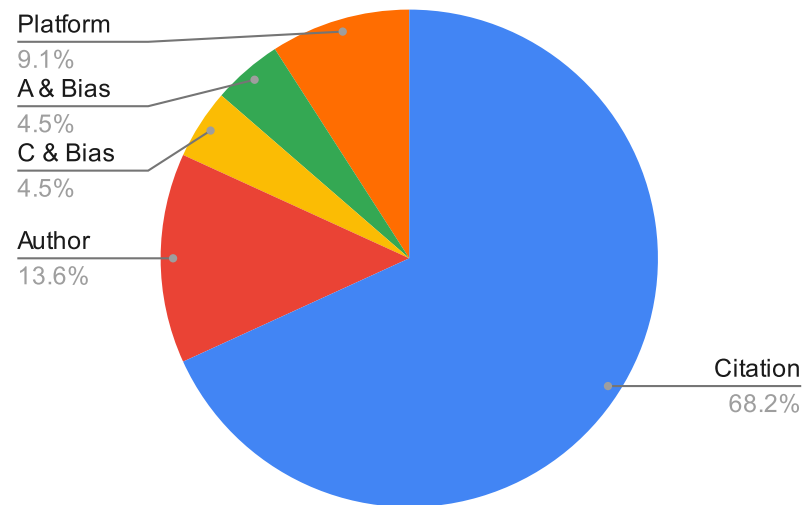
What content are you interested in the most?



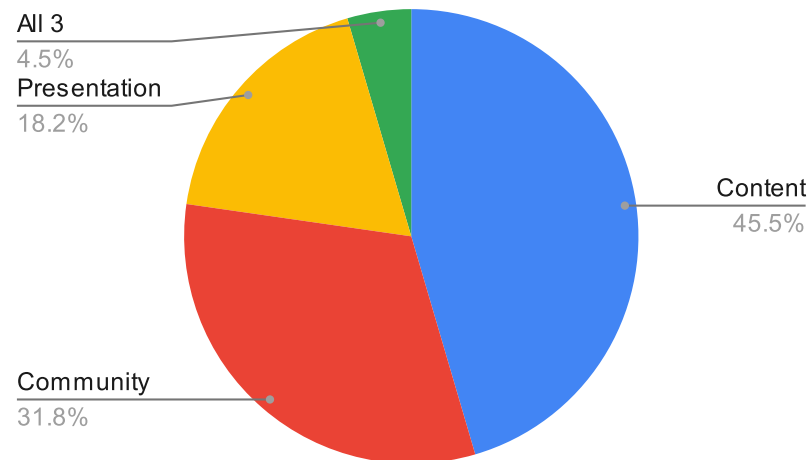
How do you usually discover topics you'd explored in-depth (rabbit holes/deep dives)?



What makes a piece of news or information credible to you?



What is the most important aspect of a community platform?



About



- Computer-science, developer-oriented from aesthetics to UX
- Content integrity focused, GitHub-inspired PR system



Scope

- Website / web platform
- In HCMUT
- Main stakeholders: students and lecturers



Related works

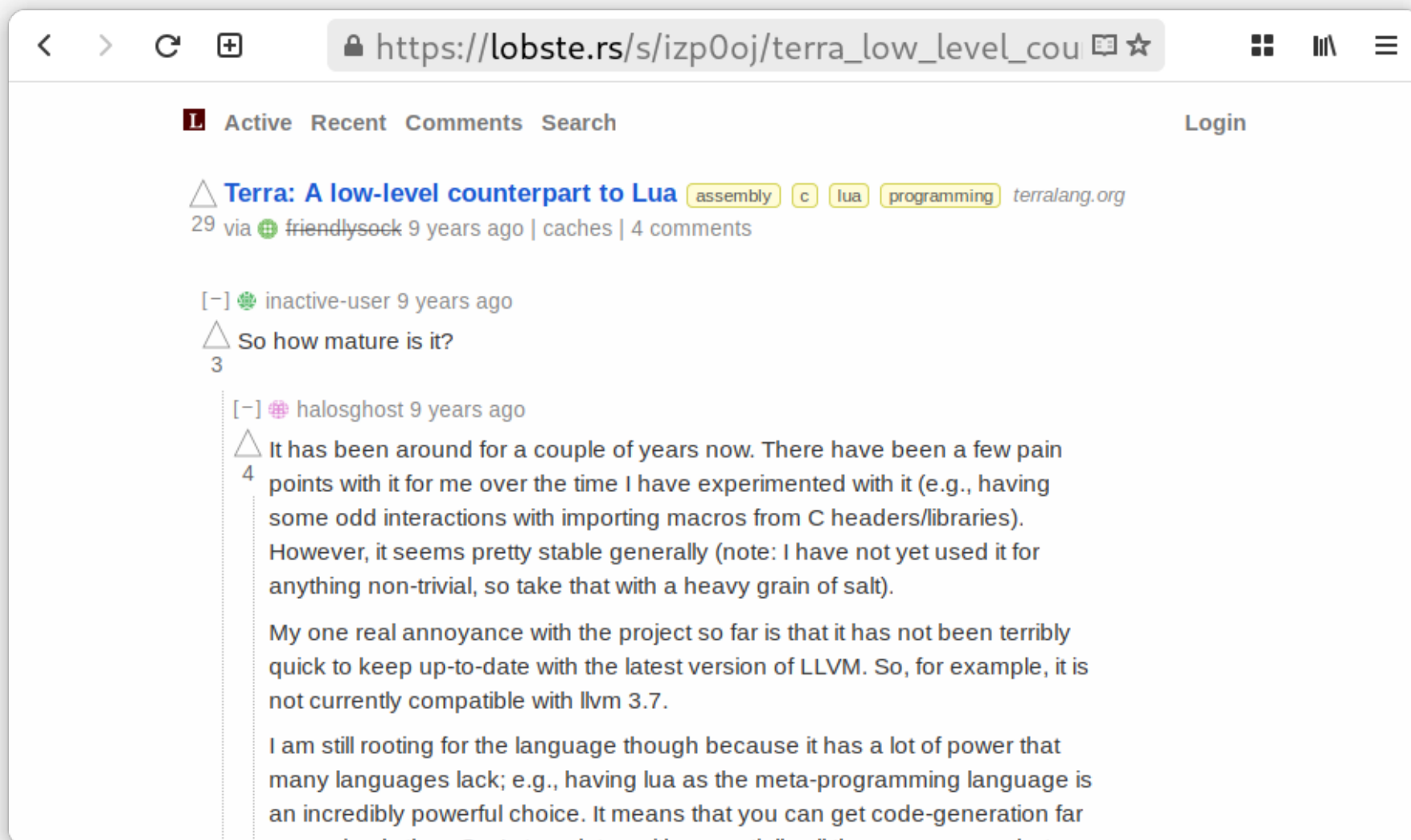
Hackernews





Lobsters





LMS Forums



https://lms.hcmut.edu.vn/mod/forum/discuss.

Trang chủ

Bảng Điều khiển

Các khoá học của tôi

BKeL Cũ

Khóa học

8

5

Hiện thị phức đáp theo kiểu cấu trúc

Các cài đặt

Về program structure

Bởi **Trần Quốc Trung** - Thứ Năm, 13 tháng 2 2025, 1:46 PM

Em chào thầy ạ !

Trong phần 2 của file mô tả về program structure, thì chương trình program chỉ bao gồm những declarations và không bao gồm các loại statement khác như assignment statement, call statement,...

Em muốn hỏi là phần này mình có cần check trong bước Parser không hay là để qua bước sau mới check. Tức là giả sử testcase của em chỉ bao gồm một statement duy nhất là call statement (không nằm trong block của khai báo hàm) thì bước Parser có check lỗi ở đây chưa ạ ?

2 Program structure



Analysis

Actors / stakeholders

- User students
- User lecturers: additional tags
- Admin

Functional requirements



ID	Description
FR1	The system supports multiple content types, including articles, discussions, issues, pull requests, revision history entries, and notes.
FR2	All content types support creation, viewing, editing, and deletion according to user permissions.
FR3	All content lists—posts, notes, issues, pull requests, revision histories, users, and feeds—support rich filtering, searching, sorting, and collapsing.
FR4	Article posts maintain complete version history, recording all edits, administrative actions, ownership transfers, and reverts.
FR5	Users can view diffs between any two article versions.
FR6	Posts allow inspecting and updating editorial permissions.
FR7	Users may create issues associated with any article post.
FR8	Users may create pull requests proposing changes to any article post.
FR9	Each issue and pull request includes its own discussion thread.
FR10	Pull requests present structured change information, including diffs, comments, and status.

Non-functional requirements

- Performance
- Reliability and availability
- Security
- Usability and accessibility
- Maintainability and accessibility
- Documentation and DX
- Community and ethics



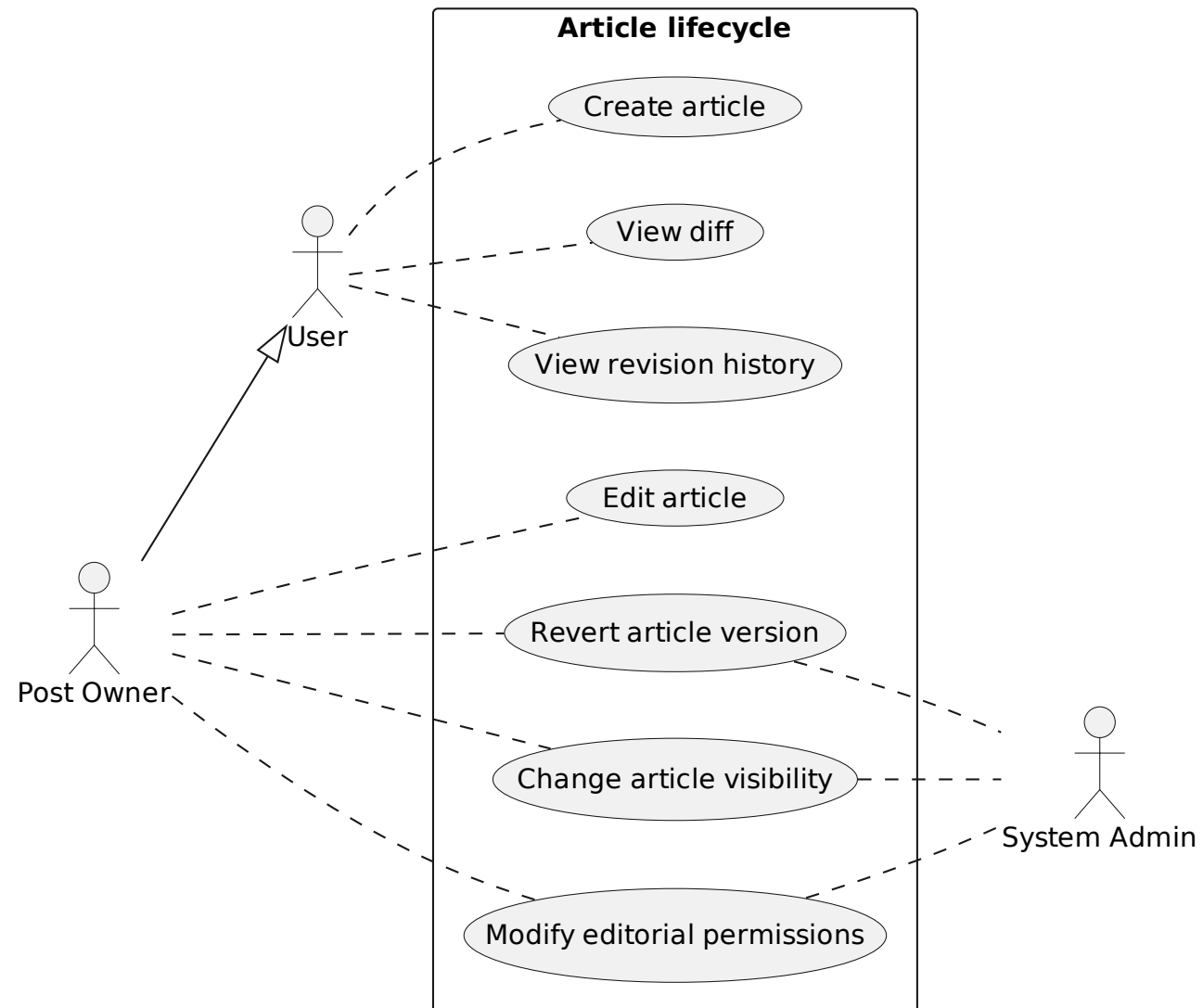
Design

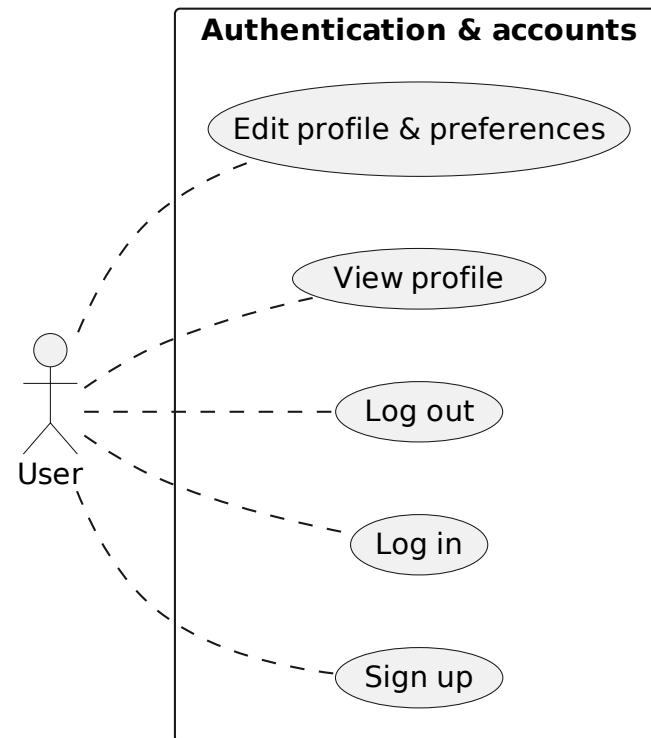
Design goals

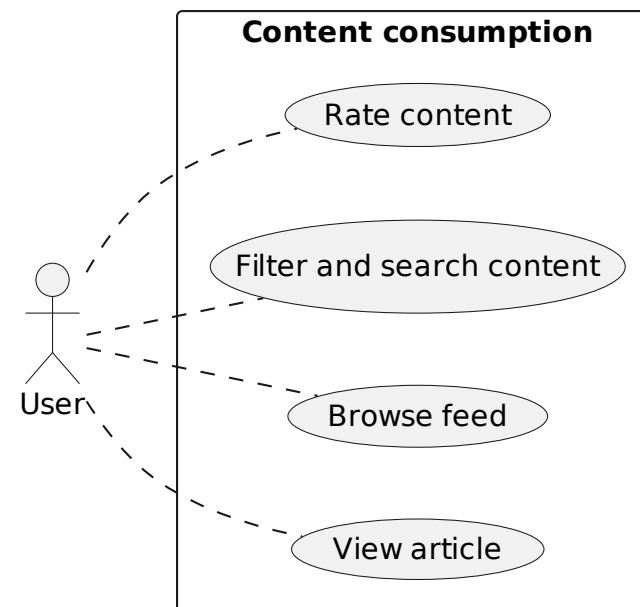
1. How to direct users to act courteously via UX?
2. How to write clean maintainable code with high performance?
3. How to minimize layering in architecture?

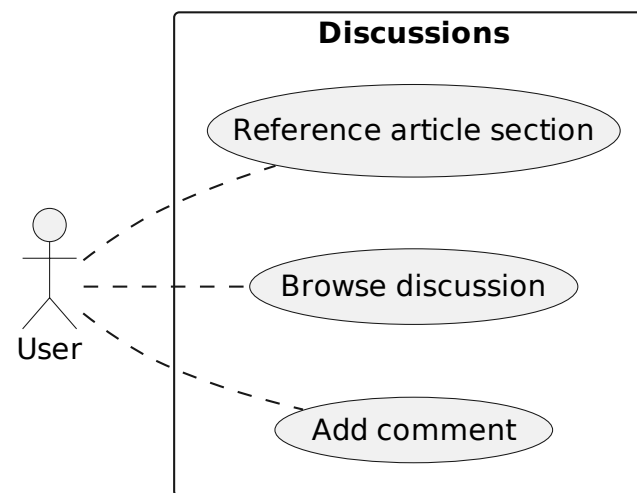
Use case

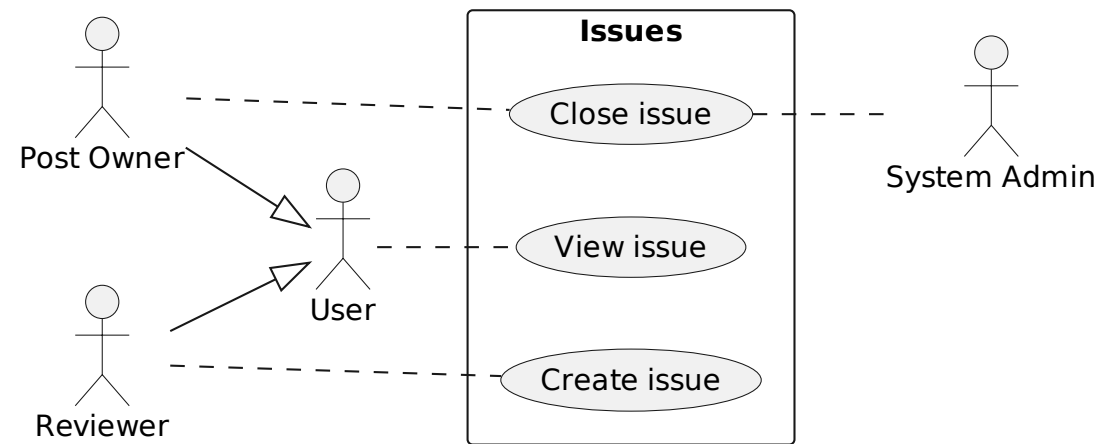
1. Article lifecycle
2. Authentication & account
3. Content consumption
4. Discussions
5. Issues
6. Pull requests
7. Moderation
8. Notes

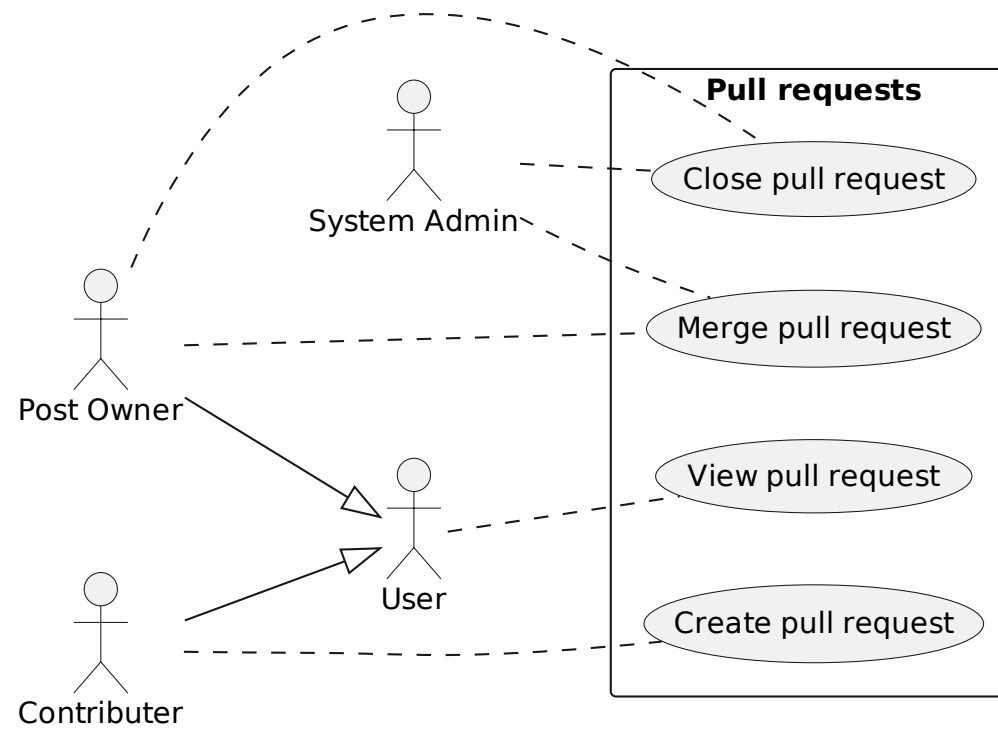


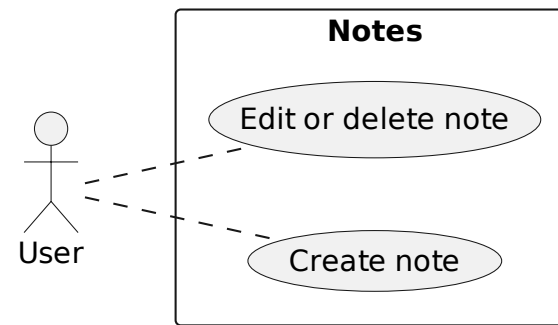


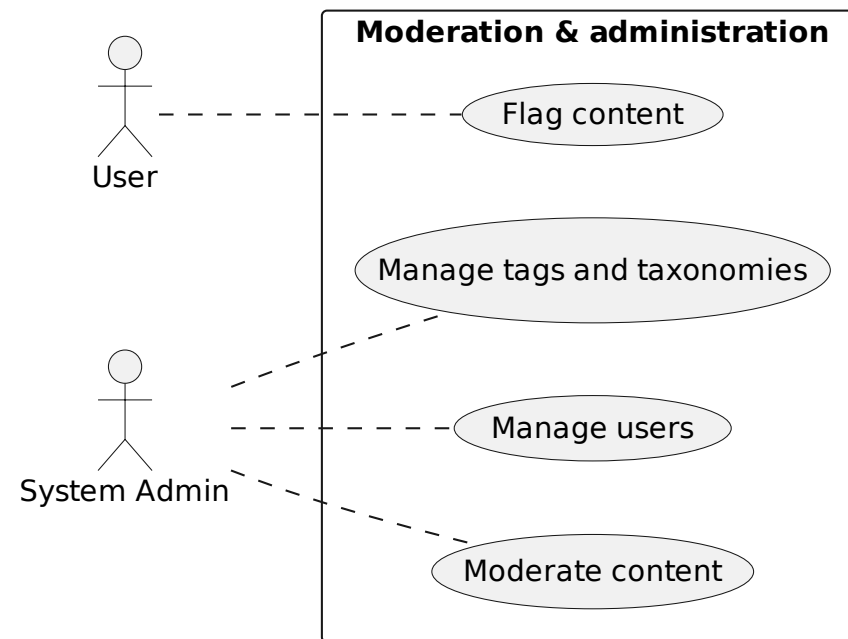














Create pull request

UC01	Pull Request	
Dependencies	Article	
Description	This use case describes how a user proposes, views, edits, and finalizes changes to an existing article through a pull request workflow. A pull request encapsulates proposed modifications, supports review via diffs and discussions, and may be approved, edited, or abandoned by its author.	
Precondition	The user is authenticated and has permission to propose changes to the target article. The target article exists and is accessible to the user.	
Ordinary Sequence	Step	Activity
	1	The user navigates to the pull requests list.
	2	The user optionally applies filters, sorting, or pagination to browse pull requests.
	3	The user selects “create pull request” and enters the pull request editor.

UC01	Pull Request	
	4	The user edits proposed changes and may preview the rendered result.
	5	The user confirms creation, and the pull request is created and stored.
	6	The user may later view an existing pull request.
	7	The user may inspect the pull request post, associated comments, or the list of changes (diffs).
	8	The author may edit the pull request and confirm the updated version.
	9	The author may approve the pull request, concluding the process.
Post Condition	A pull request is created, updated, or approved. Its state and associated discussions and diffs are persisted and become visible according to permissions.	
Exceptions	Step	Activity
	1	The user cancels pull request creation, and no changes are saved.
	2	The user lacks sufficient permissions, and the action is denied.
Comments	This use case focuses on the author-side lifecycle. Administrative approval, rejection, or merging is covered in a separate moderation or merge use case.	

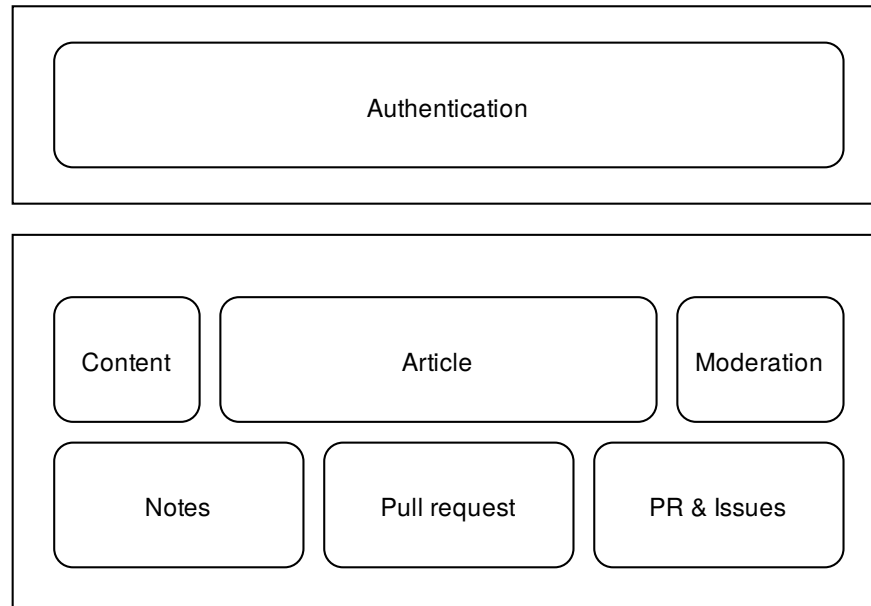
System architecture

- Service-oriented architecture (SOA)
- (in next page)

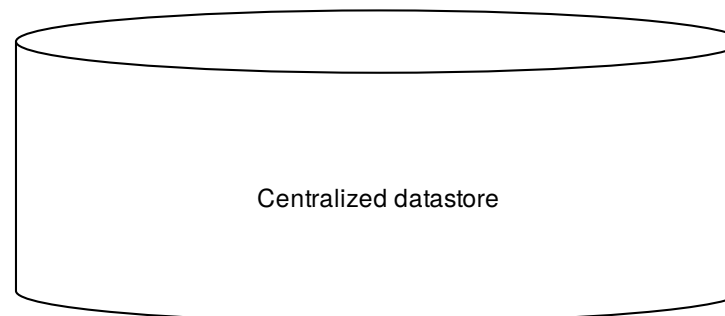
Presentation tier
(Front-end)



Business tier
(Back-end)



Data store tier
(Database)

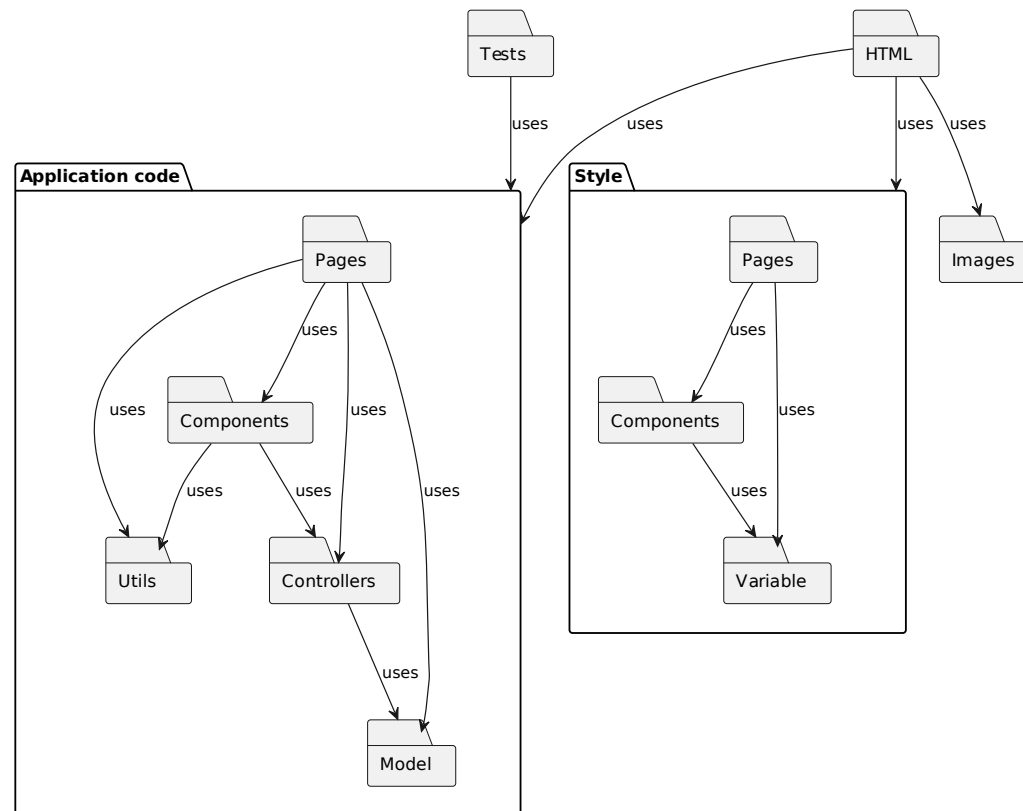




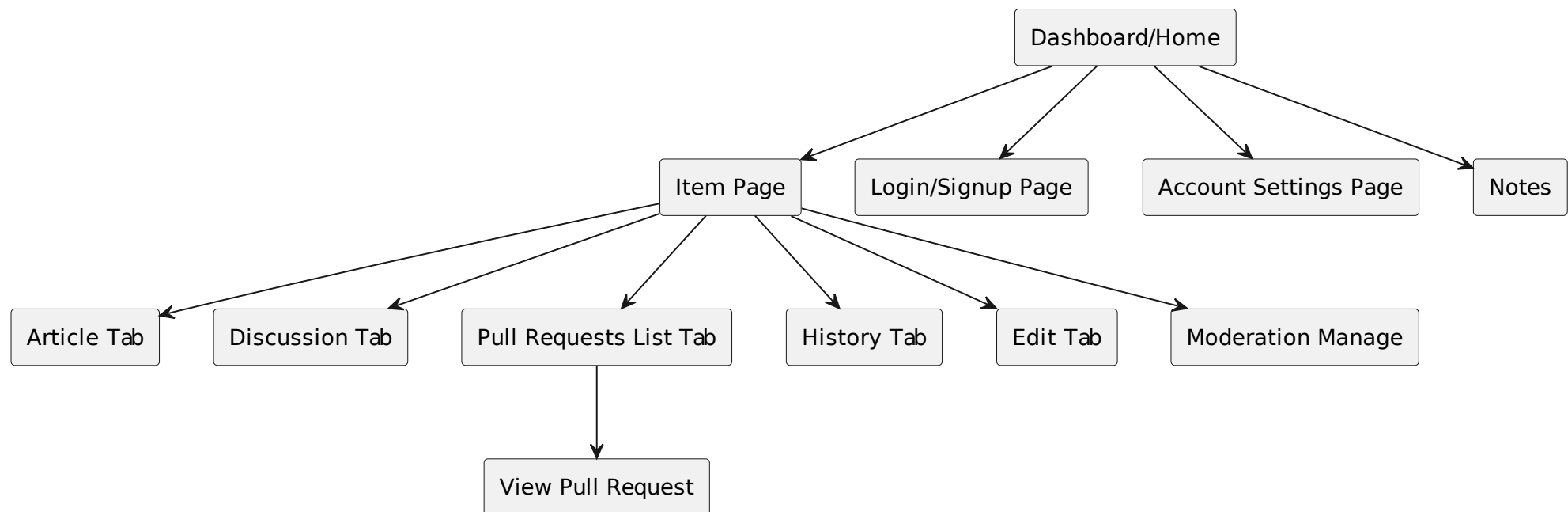
Front-end

- Model-view-controller (MVC)

Module view (“use” diagram)



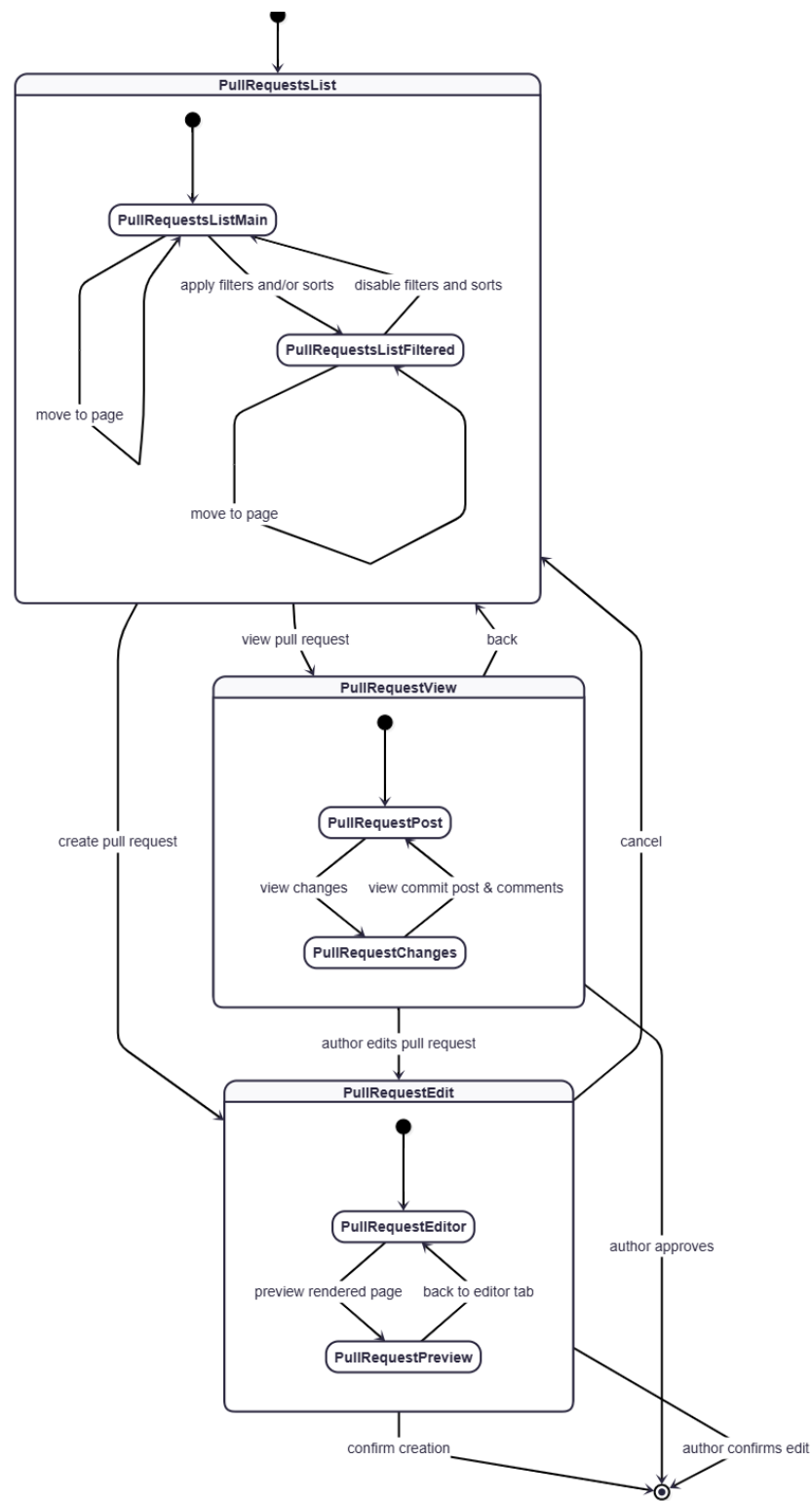
Sitemap





Pull request states

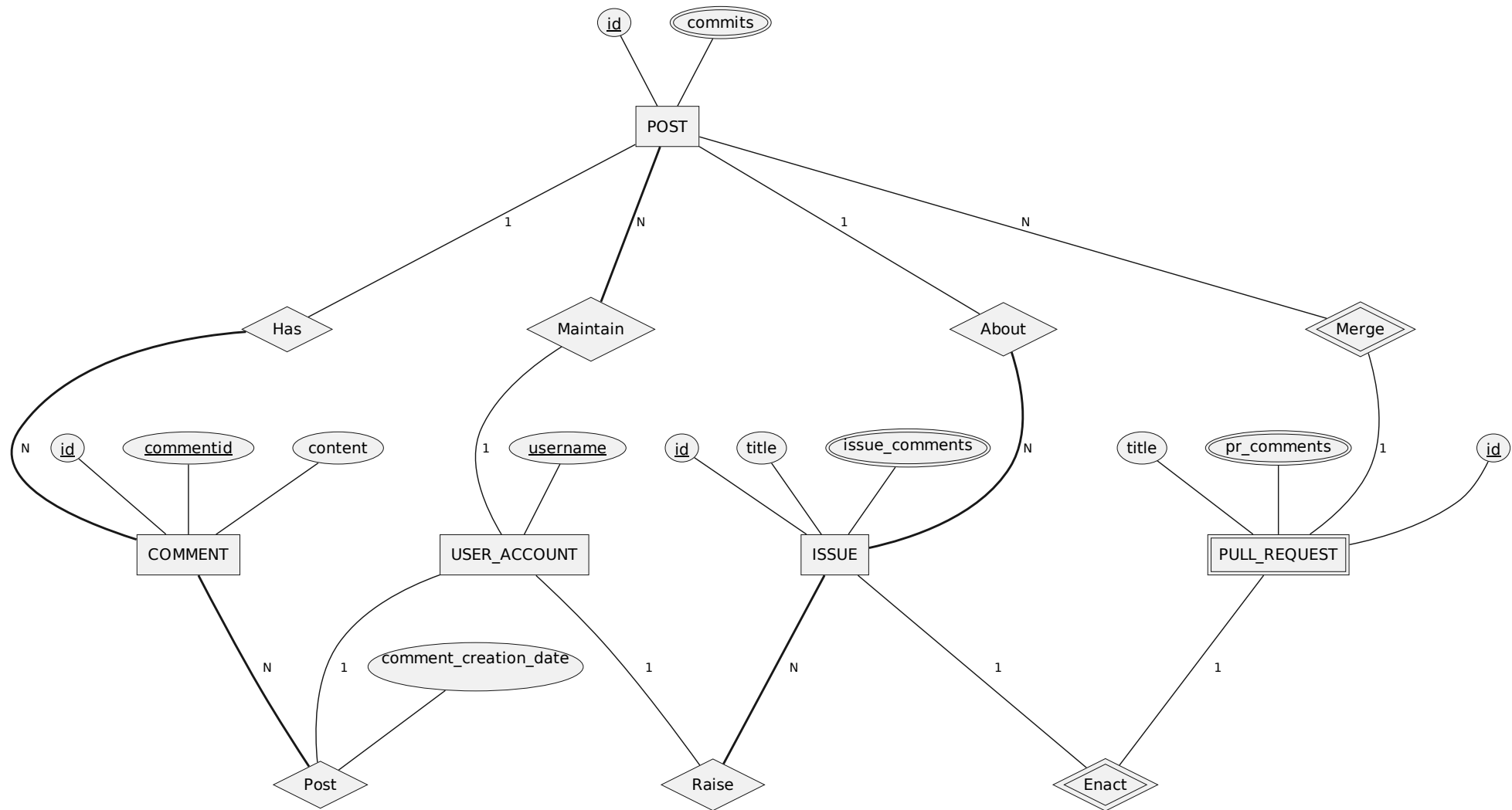
(in next page)





ERD

(in next page)



User interface prototypes

(in next pages)

Home



BKHack

\$ search [posts | tags | users]...

home feed

\$ feed --sort=hot --filter

\$ grep -i '...'

hot

25/page

page

1 / 42

342

Understanding the Time Complexity of Graph Algorithms

#AlgGtCa

commented @wilson 2h ago

5d ago

ref:2009

287

Dynamic Programming vs Greedy Algorithms: When to Use Which

#AlgDpOpt

edited @kim 4h ago

2d ago

ref:2023

421

Rust Memory Safety Without Garbage Collection Explained

#RusMmSp

merged_pr @rodriguez 1d ago

12d ago

ref:2024

198

A* Search Algorithm: Heuristics and Implementation

#AlgSeaA1

opened_pr @zhang 3d ago

1w ago

ref:1968

312

Understanding Concurrency in Golang with Goroutines

#GolConPer

commented @chen 5h ago

4d ago

ref:2022

156

Blockchain Consensus Mechanisms: PoW vs PoS vs PBFT

#BloBsCry

opened_issue @wang 6h ago

3d ago

ref:2008

prev

page

1

2

...

52

next

of 52 pages total

[hide]

[my_notes]

Amortized analysis key insight

Fibonacci heap practical considerations

+ new note

[search]

\$ search posts...

\$ author:username

\$ tag:algorithms

search

[quick_filters]

☐ show_unseen_only

☐ subscribed_only

☐ modified_only

☒ verified_only

[trending_tags]

#algorithms

#golang

#rust

#performance

#distributed-sys

#ai-ml

Post



BKHack

\$

search [posts | tags | users]...

article.md

discussions [24]

issues [3]

pull_requests [2]

git log

vim

chmod

[hide]

↑

Understanding the Time Complexity of Graph Algorithms

330

↓

#algorithms #graph-theory #complexity #cs-theory

dr.maria.chen

[lecturer]

modified: 2d

created: 5d

PRs: 2

issues: 3

watch: on

read

subscribe

> [contents]

1. overview

2. algorithms

2.1 bfs_dfs

2.2 dijkstra

2.3 floyd_warshall

3. data_structures

4. practical

5. references

In computer science, **graph algorithm complexity analysis** is the study of the computational resources required to solve problems on graph data structures. This article provides a comprehensive examination of time complexity for fundamental graph algorithms, including breadth-first search (BFS), depth-first search (DFS), Dijkstra's algorithm, and the Floyd-Warshall algorithm.

Understanding these complexities is essential for algorithm selection and optimization in practical applications ranging from network routing to social network analysis.

overview

Graph algorithms operate on graph data structures $G = (V, E)$, where V represents the set of vertices and E represents the set of edges. The time complexity of these algorithms is typically expressed in terms of $|V|$ (number of vertices) and $|E|$ (number of edges).

The choice of algorithm depends on several factors including graph density, presence of negative weights, and whether single-source or all-pairs shortest paths are required.

common_algorithms

bfs_and_dfs

Both BFS and DFS are fundamental graph traversal algorithms with a time complexity of $O(V + E)$. These algorithms visit each vertex and edge exactly once, making them optimal for graph exploration tasks.

time_complexity: $O(V + E)$

[my_notes]

Amortized analysis key insight

Fibonacci heap practical considerations

+ new note

[stats]

views 1,234

score +330

discussions 24

issues 3

pull_requests 2

[author]

dr.maria.chen

Maria Chen, PhD

[lecturer]

posts 47

contributions 312

reputation 8,456

[timeline]

created

46



Realization

Technologies

- Front-end: Reason & React
- Back-end: Elixir
- Front-end Hosting: Firebase
- Back-end Hosting: Render; Google Cloud
- Database: Firestore

Inline markup

Writing layout markup in same place as application code

```
let makeComponent = () => {  
  let message = "hi";  
  <div className="label">  
    {message}  
  </div>  
}
```

Self-documentation

- Incentivize writing documentations
- Possibly subjective



Other

- Design system



Timeline

Break down tasks

- Based on use cases
- Based on crosscutting / infra / nonfunctional

Module	Task	Developer
Auth. & acc.	Sign up & Log in	Bao
	View account info	Khang
	Edit preferences	Tuong
Content consumption	View article	Bao
	List articles, filter feed, search	Khang
	Rate	Tuong
Article lifecycle	Article view, create, edit	Bao
	History, diff, log	Khang
	Configure article and permissions	Tuong

Table 19: Phase one

Module	Task	Developer
Discussion	Create and reply comments	Bao
	Filter and search comments	Khang
	Article referencing	Tuong
Issues	Create issue	Bao
	View issue, comments	Khang
	Close issue	Tuong
Pull requests	Create issue	Bao
	View pull requests and comments	Khang
	Merge, close pull request	Tuong

Table 20: Phase two

Module	Task	Developer
Notes	Create Note	Bao
	Edit Note	Khang
	Delete Note	Tuong
Moderation	Flag and review content	Bao
	Manage Users	Khang
	Manage tags	Tuong

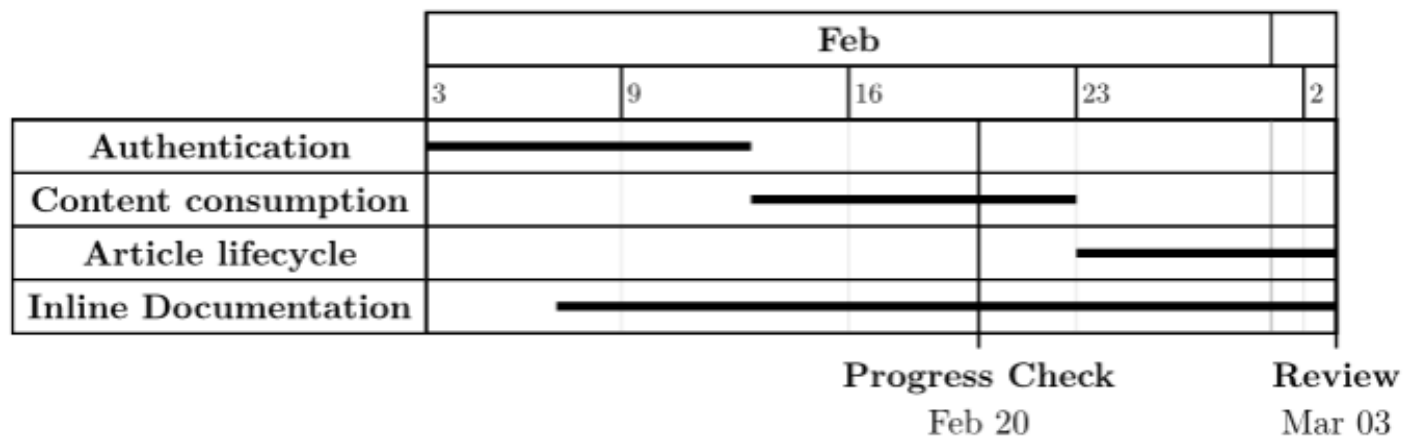
Table 21: Phase three

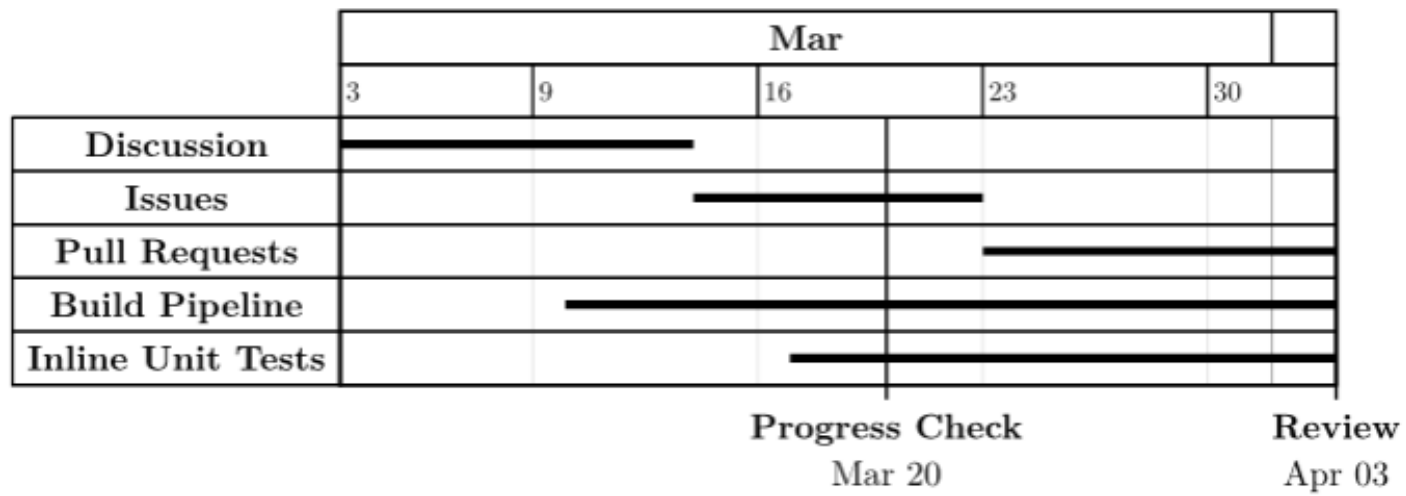
Task	Developer
Inline and Document-based Documentation	Bao
Inline Unit Tests	Khang
Build Pipeline	Tuong
Functional Testing	Khang
Service Testing	Bao

Table 22: Cross-cutting

Schedule tasks

- 4 months
- Each module ~1.5 weeks (~1/3 month)
- Final month is for testing





	Apr					
	3	6	13	20	27	
Notes						
Moderation						
Functional Testing						
Service Testing						
Progress Check						Review
Apr 20						May 03

	May					
	3	4	11	18	25	1
Functional Testing						
Service Testing						

Progress Check
May 20

Conclusion

- Finished design
- Running web application
- Next phase planned



Thank you!
